

Computer Science Fundamentals

Comprehensive Study Guide

Operating Systems | Computer Networks | DBMS | OOPs | DSA

Prepared by Gaurang Khator

May 2026

Operating Systems (OS)

Introduction and History

[illegible]

[illegible]

reveals that systematic implementation requires careful balancing of memory, speed, and reliability.

[illegible]

Process Management

[illegible]

[illegible]

[illegible]

tasks like creation, scheduling, termination of processes, and a dead lock. The OS manages processes' information such as State, Program Counter, CPU registers, CPU scheduling information, Memory-management information, Accounting information, and I/O status information. A process is a program in execution. Process management involves various tasks like creation, scheduling, termination of processes, and a dead lock. The OS manages processes' information such as State, Program Counter, CPU registers, CPU scheduling information, Memory-management information, Accounting information, and I/O status information. A process is a program in execution. Process management involves various tasks like creation, scheduling, termination of processes, and a dead lock. The OS manages processes' information such as State, Program Counter, CPU registers, CPU scheduling information, Memory-management information, Accounting information, and I/O status information.

[illegible][illegible]

efficient architecture. remains a cornerstone of efficient architecture. remains a
cornerstone of efficient architecture. remains a cornerstone of efficient architecture.
remains a cornerstone of efficient architecture. remains a cornerstone of efficient
architecture. remains a cornerstone of efficient architecture. remains a cornerstone of
efficient architecture.

CPU Scheduling

[illegible]

[illegible]

[illegible]

[illegible][illegible]

Memory Management

Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution. It keeps track of each and every memory location, regardless of whether it is allocated to some process or it is free. Memory management is the

[illegible]

[illegible]

memory location, regardless of whether it is allocated to some process or it is free. Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution. It keeps track of each and every memory location, regardless of whether it is allocated to some process or it is free. Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution. It keeps track of each and every memory location, regardless of whether it is allocated to some process or it is free. Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution. It keeps track of each and every memory location, regardless of whether it is allocated to some process or it is free. Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution. It keeps track of each and every memory location, regardless of whether it is allocated to some process or it is free.

[illegible]

[illegible]

Deadlocks

[illegible]

[illegible]

Conditions for deadlock: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait. A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

Conditions for deadlock: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait. A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

Conditions for deadlock: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait. A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

Conditions for deadlock: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait. A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

Conditions for deadlock: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait. A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

Conditions for deadlock: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait. A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

Wait.

[illegible]

[illegible]

[illegible]

implementation requires careful balancing of memory, speed, and reliability. reveals that systematic implementation requires careful balancing of memory, speed, and reliability.

[illegible]

TCP/IP Protocol Suite

[illegible]

[illegible]

Network Topologies

[illegible][illegible]

[illegible][illegible]

Network Security

Network security consists of the policies, processes and practices adopted to prevent, detect and monitor unauthorized access, misuse, modification, or denial of a computer network and network-accessible resources. Network security consists of the policies, processes and practices adopted to prevent, detect and monitor unauthorized access, misuse, modification, or denial of a computer network and network-accessible resources. Network security consists of the policies, processes and practices adopted to prevent, detect and monitor unauthorized access, misuse, modification, or denial of a computer network and network-accessible resources. Network security consists of the policies, processes and practices adopted to prevent, detect and monitor unauthorized access, misuse, modification, or denial of a computer network and network-accessible resources. Network security consists of the policies, processes and practices adopted to prevent, detect and monitor unauthorized access, misuse, modification, or denial of a computer network and network-accessible resources.

[illegible]

[illegible][illegible]

In modern computing environments, Network Security remains a cornerstone of efficient architecture. remains a cornerstone of efficient architecture. remains a cornerstone of efficient architecture. remains a cornerstone of efficient architecture. remains a

cornerstone of efficient architecture. remains a cornerstone of efficient architecture.
remains a cornerstone of efficient architecture. remains a cornerstone of efficient
architecture. remains a cornerstone of efficient architecture. remains a cornerstone of
efficient architecture. remains a cornerstone of efficient architecture. remains a
cornerstone of efficient architecture. remains a cornerstone of efficient architecture.
remains a cornerstone of efficient architecture. remains a cornerstone of efficient
architecture.

Database Management Systems (DBMS)

Introduction to DBMS

[illegible]

[illegible]

create, read, update, and delete data in a database. It serves as an interface between the database and its end-users or programs.

[illegible][illegible]

ER Modeling

[illegible]

[illegible]

[illegible]

Relational Model and SQL

[illegible]

[illegible]

Technical deep-dive into Relational Model and SQL reveals that systematic implementation requires careful balancing of memory, speed, and reliability. reveals that systematic implementation requires careful balancing of memory, speed, and reliability. reveals that systematic implementation requires careful balancing of memory, speed, and reliability. reveals that systematic implementation requires careful balancing of memory, speed, and reliability. reveals that systematic implementation requires careful balancing of memory, speed, and reliability.

speed, and reliability. reveals that systematic implementation requires careful balancing
of memory, speed, and reliability. reveals that systematic implementation requires
careful balancing of memory, speed, and reliability. reveals that systematic
implementation requires careful balancing of memory, speed, and reliability. reveals that
systematic implementation requires careful balancing of memory, speed, and reliability.
reveals that systematic implementation requires careful balancing of memory, speed, and
reliability. reveals that systematic implementation requires careful balancing of memory,
speed, and reliability. reveals that systematic implementation requires careful balancing
of memory, speed, and reliability. reveals that systematic implementation requires
careful balancing of memory, speed, and reliability. reveals that systematic
implementation requires careful balancing of memory, speed, and reliability. reveals that
systematic implementation requires careful balancing of memory, speed, and reliability.
reveals that systematic implementation requires careful balancing of memory, speed, and
reliability.

[illegible]

Normalization

Normalization is a process of organizing data in a database to reduce redundancy and improve data integrity. Forms include 1NF, 2NF, 3NF, and BCNF. Normalization is a process of organizing data in a database to reduce redundancy and improve data integrity. Forms include 1NF, 2NF, 3NF, and BCNF. Normalization is a process of organizing data in a database to reduce redundancy and improve data integrity. Forms include 1NF, 2NF, 3NF, and BCNF. Normalization is a process of organizing data in a database to reduce redundancy and improve data integrity. Forms include 1NF, 2NF, 3NF, and BCNF. Normalization is a process of organizing data in a database to reduce redundancy and improve data integrity. Forms include 1NF, 2NF, 3NF, and BCNF.

[illegible]

and BCNF. Normalization is a process of organizing data in a database to reduce redundancy and improve data integrity. Forms include 1NF, 2NF, 3NF, and BCNF.

[illegible]

In modern computing environments, Normalization remains a cornerstone of efficient architecture.

Transaction Management

remains a cornerstone of efficient architecture. remains a cornerstone of efficient architecture. remains a cornerstone of efficient architecture. remains a cornerstone of efficient architecture.

Object Oriented Programming (OOPs)

Encapsulation

[illegible]

[illegible][illegible]

careful balancing of memory, speed, and reliability. reveals that systematic implementation requires careful balancing of memory, speed, and reliability. reveals that systematic implementation requires careful balancing of memory, speed, and reliability.

[illegible]

Inheritance

[illegible]

[illegible][illegible]

Polymorphism

Polymorphism allows objects of different classes to be treated as objects of a common superclass. It includes compile-time polymorphism (overloading) and runtime polymorphism (overriding). Polymorphism allows objects of different classes to be treated as objects of a common superclass. It includes compile-time polymorphism (overloading) and runtime polymorphism (overriding). Polymorphism allows objects of different classes to be treated as objects of a common superclass. It includes compile-time polymorphism (overloading) and runtime polymorphism (overriding). Polymorphism allows objects of different classes to be treated as objects of a common superclass. It includes compile-time polymorphism (overloading) and runtime polymorphism (overriding). Polymorphism allows objects of different classes to be treated as objects of a common superclass. It includes compile-time polymorphism (overloading) and runtime polymorphism (overriding).

[illegible]

(overloading) and runtime polymorphism (overriding). Polymorphism allows objects of different classes to be treated as objects of a common superclass. It includes compile-time polymorphism (overloading) and runtime polymorphism (overriding). Polymorphism allows objects of different classes to be treated as objects of a common superclass. It includes compile-time polymorphism (overloading) and runtime polymorphism (overriding). Polymorphism allows objects of different classes to be treated as objects of a common superclass. It includes compile-time polymorphism (overloading) and runtime polymorphism (overriding).

[illegible][illegible]

remains a cornerstone of efficient architecture. remains a cornerstone of efficient architecture.

Abstraction

[illegible]

implementation requires careful balancing of memory, speed, and reliability. reveals that systematic implementation requires careful balancing of memory, speed, and reliability.

[illegible]

Classes and Objects

[illegible]

[illegible][illegible]

In modern computing environments, Classes and Objects remains a cornerstone of efficient architecture. remains a cornerstone of efficient architecture. remains a

cornerstone of efficient architecture. remains a cornerstone of efficient architecture.
remains a cornerstone of efficient architecture. remains a cornerstone of efficient
architecture. remains a cornerstone of efficient architecture. remains a cornerstone of
efficient architecture. remains a cornerstone of efficient architecture. remains a
cornerstone of efficient architecture. remains a cornerstone of efficient architecture.
remains a cornerstone of efficient architecture. remains a cornerstone of efficient
architecture. remains a cornerstone of efficient architecture. remains a cornerstone of
efficient architecture.

[illegible]

[illegible][illegible]

[illegible]

Linked Lists

[illegible]

efficient architecture. remains a cornerstone of efficient architecture. remains a
cornerstone of efficient architecture. remains a cornerstone of efficient architecture.
remains a cornerstone of efficient architecture. remains a cornerstone of efficient
architecture. remains a cornerstone of efficient architecture. remains a cornerstone of
efficient architecture. remains a cornerstone of efficient architecture. remains a
cornerstone of efficient architecture. remains a cornerstone of efficient architecture.
remains a cornerstone of efficient architecture. remains a cornerstone of efficient
architecture.

Trees and Graphs

[illegible]

[illegible][illegible]

[illegible]

Sorting and Searching

[illegible]

implementation requires careful balancing of memory, speed, and reliability. reveals that systematic implementation requires careful balancing of memory, speed, and reliability.

[illegible]

Dynamic Programming

[illegible]

[illegible]

down into simpler subproblems and storing the results of subproblems to avoid redundant computations.

[illegible][illegible]